

Cloud Composer (Apache Airflow on GCP) — A Beginner-to-Practitioner Handbook

1. What Is Cloud Composer?

Cloud Composer is Google Cloud's **fully managed workflow orchestration service**, built on top of the open-source project **Apache Airflow**.

Think of it this way:

- **Apache Airflow** is the open-source engine that lets you author, schedule, and monitor workflows written as Python code.
- **Cloud Composer** is Google's managed wrapper around Airflow — it provisions and maintains the Airflow infrastructure (database, scheduler, web server, workers) on top of **Google Kubernetes Engine (GKE)**, so you don't have to install, patch, or scale Airflow yourself.

Why it exists: Data pipelines rarely consist of a single script. A real pipeline might need to: wait for a file to land in Cloud Storage → load it into BigQuery → run a transformation → trigger a Dataflow job → send a Slack notification if something fails → retry a step 3 times before giving up. Doing this reliably, with retries, dependency management, and visibility, is what Airflow/Composer is built for.

Core idea: In Airflow, a workflow is a **DAG** (Directed Acyclic Graph) — a Python file that describes *what* steps to run and *in what order*, not the actual heavy-lifting logic itself (that usually happens in BigQuery,

Dataflow, Dataproc, etc., which Airflow just triggers and monitors).

2. Core Airflow Concepts and Terminology

This is the vocabulary you must be fluent in before writing your first DAG.

Term	Plain-English Explanation
DAG (Directed Acyclic Graph)	A workflow definition — a Python file describing a set of tasks and the order they must run in. "Directed" = arrows point one way (task A → task B). "Acyclic" = no loops; you can't go back to a task you already completed.
Task	A single unit of work inside a DAG (e.g., "load file to BigQuery"). Represented as a node in the DAG graph.
Task Instance	One specific run of a task, tied to a specific execution date/time. The same task run on different days produces different Task Instances.
Operator	A Python class that defines <i>what kind</i> of work a task does. Airflow ships pre-built operators, e.g. <code>BashOperator</code> (runs a shell command), <code>PythonOperator</code> (runs a Python function), <code>BigQueryInsertJobOperator</code> (runs a BigQuery query), <code>GCSToBigQueryOperator</code> (loads a Cloud Storage file into BigQuery). You instantiate an operator to create a task.
Sensor	A special type of operator that <i>waits</i> for something to become true before letting downstream tasks run — e.g. <code>GCSObjectExistenceSensor</code> waits until a file appears in a bucket.

Hook	A reusable interface for talking to an external system (e.g. <code>BigQueryHook</code> , <code>GCSHook</code>). Operators use hooks under the hood; you can also use hooks directly inside a <code>PythonOperator</code> for custom logic.
Scheduler	The Airflow component that continuously scans your DAG files, figures out which tasks are due to run based on their schedule and dependencies, and queues them.
Executor	The component that actually decides <i>how and where</i> queued tasks get run (e.g. <code>CeleryExecutor</code> , <code>KubernetesExecutor</code>). Composer 2 defaults to the <code>CeleryKubernetesExecutor</code> or <code>KubernetesExecutor</code> depending on configuration.
Worker	The process that executes the actual task code. In Composer, workers run as pods inside the underlying GKE cluster.
Webserver (UI)	The browser-based dashboard where you view DAGs, trigger runs manually, inspect logs, and see task status (green = success, red = failed, yellow = running/up-for-retry, etc.).
Metadata Database	A database (Cloud SQL, in Composer's case) that stores DAG run history, task states, variables, connections, and XComs. This is Airflow's "memory."
Execution Date / Logical Date	The timestamp a DAG run is <i>logically</i> associated with — confusingly, it is the start of the interval being processed, not the moment the DAG actually runs. E.g., a daily DAG scheduled for 2026-07-15 with <code>schedule_interval='@daily'</code> actually runs after 2026-07-15 23:59 ends, because Airflow waits for the full interval to complete first.

Schedule Interval	How often a DAG should run — a cron expression (<code>'0 6 * * *'</code>) or a preset (<code>@daily</code> , <code>@hourly</code> , <code>@once</code> , <code>None</code> for manual-trigger-only).
Catchup	If <code>catchup=True</code> (the default), Airflow will automatically run a DAG for every schedule interval it "missed" between <code>start_date</code> and today. Almost always set <code>catchup=False</code> for training/demo DAGs to avoid a flood of backfilled runs.
Backfill	Manually or automatically running a DAG for past dates it hasn't yet processed.
XCom (Cross-Communication)	A small key-value mechanism that lets one task pass a small piece of data to another (e.g., a task pushes a BigQuery job ID, and a downstream task pulls it). Not meant for large data — just metadata/pointers.
Variable	A key-value pair stored in Airflow's metadata DB, used for configuration values shared across DAGs (e.g., a GCS bucket name). Set via UI, CLI, or <code>gcloud composer environments run</code> .
Connection	A stored set of credentials/config for connecting to an external system (e.g., a BigQuery project, an SSH server, a Slack webhook). In Composer, the default <code>google_cloud_default</code> connection is pre-configured to use the environment's service account.
Pool	A mechanism to limit how many tasks of a certain kind run concurrently (e.g., cap concurrent connections to a rate-limited API).

Trigger Rule	Controls when a task should fire based on the state of its upstream tasks. Default is <code>all_success</code> (all parents must succeed). Others: <code>all_failed</code> , <code>one_success</code> , <code>none_failed</code> , <code>all_done</code> — useful for building "run this cleanup step regardless of success/failure" logic.
SLA (Service Level Agreement)	An optional time limit on a task; if exceeded, Airflow can alert you.
Dependency (>> / <<)	The operators used to chain tasks: <code>task_a >> task_b</code> means <code>task_b</code> runs only after <code>task_a</code> succeeds.
DAG Run	One complete execution instance of an entire DAG (as opposed to a single task instance).
Idempotency	A design principle, not an Airflow feature: a task should produce the same result if re-run for the same logical date. Critical because Airflow retries and backfills re-execute tasks.

3. Cloud Composer—Specific Concepts

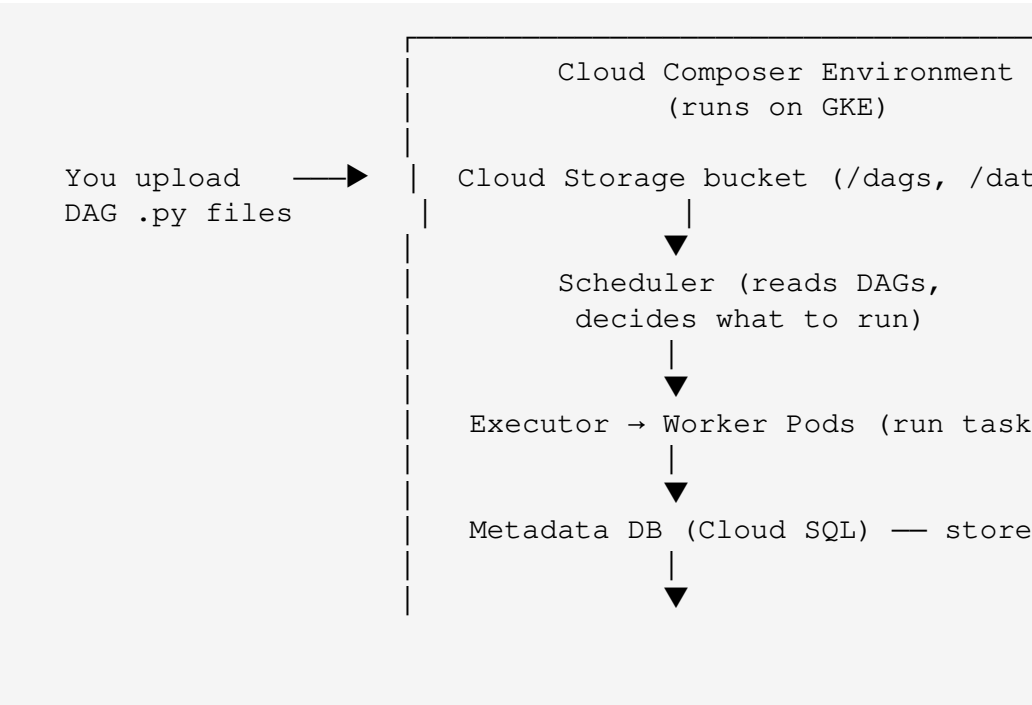
Beyond vanilla Airflow, Composer adds GCP-managed infrastructure and its own vocabulary:

Term	Explanation
Environment	A single deployed instance of Composer — essentially "your Airflow cluster." Each environment has its own GKE cluster, Cloud SQL metadata DB, Cloud Storage bucket, and web server. You create environments per project/team/use case.

Composer 1 vs Composer 2	Composer 1 used a fixed-size GKE cluster. Composer 2 (current generation) is built on GKE Autopilot-style architecture with independent, autoscaling worker pools, letting you scale schedulers, workers, and web server compute independently and scale to zero-ish when idle — significantly more cost-efficient. New environments should default to Composer 2 (or Composer 3, GCP's newest generation with further-simplified networking and autoscaling) — verify current availability, since GCP updates versions frequently.
Environment's GCS Bucket	Every Composer environment auto-creates a Cloud Storage bucket (e.g., <code>gs://<region>-<env-name>-<hash>-bucket</code>). This bucket has a <code>/dags</code> folder — this is the folder you upload your DAG Python files to, and Composer syncs them into the Airflow scheduler automatically within a few minutes. There are also <code>/data</code>, <code>/logs</code>, and <code>/plugins</code> folders.
Image Version	Composer bundles a specific Airflow version + Python version + pre-installed provider packages into an "image version" string, e.g. <code>composer-2.10.2-airflow-2.9.3</code>. You choose this at environment creation.
Provider Packages	Airflow's GCP integrations (BigQuery, Dataflow, Dataproc, Pub/Sub, GCS, etc. operators/hooks) ship as a separate installable package: <code>apache-airflow-providers-google</code>. Composer pre-installs this.
PyPI Packages	Custom Python dependencies your DAGs need (e.g., <code>pandas</code>, <code>requests</code>) — installed per-environment via the Composer UI/CLI, not via <code>pip install</code> inside a DAG file.
Environment Size (Small/Medium/Large)	A Composer 2 setting controlling the baseline scheduler/webserver/worker resource allocation.

Private IP Environment	A networking mode where the GKE cluster and Cloud SQL instance have no public IPs — standard for enterprise/production setups.
Airflow DB Cleanup / Log Retention	Composer environments accumulate metadata and logs over time; GCP recommends periodic cleanup DAGs or retention policies to control Cloud SQL and GCS storage costs.
Service Account	Every Composer environment runs under a GCP service account (default or customer-managed) that determines what GCP resources (BigQuery datasets, GCS buckets, Dataflow jobs, etc.) the DAGs are authorized to touch. This is why you usually don't need to manage separate credentials inside DAGs — <code>google_cloud_default</code> connection uses this identity automatically.

4. Architecture at a Glance



Web Server (Airflow UI)

Tasks call out to:
BigQuery, Dataflow,
GCS, Pub/Sub, etc.

You view results at:
<https://cloud.google.com/composer/monitoring>
webserver-url

5. Getting Hands-On: Step-by-Step

Step 1 — Create a Composer environment

Via Console: **Navigation Menu** → **Composer** → **Create Environment**. Choose Composer 2/3, pick a region, environment size (Small is fine for learning), and an Airflow image version.

Via `gcloud`:

```
gcloud composer environments create my-first-composer-env \
  --location us-central1 \
  --image-version composer-2.10.2-airflow-2.9.3
```

This takes ~20–25 minutes to provision — a good time to review the terminology table above.

Step 2 — Write your first DAG

Save this as `hello_composer.py`:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
```



```

default_args = {
    "owner": "himanshu",
    "retries": 1,
    "retry_delay": timedelta(minutes=2),
}

def greet():
    print("Hello from a PythonOperator task running inside")

with DAG(
    dag_id="hello_composer",
    description="A minimal DAG to verify the environment",
    default_args=default_args,
    schedule_interval="@daily",    # runs once a day
    start_date=datetime(2026, 7, 1),
    catchup=False,                # don't backfill past runs
    tags=["training", "demo"],
) as dag:

    task_start = BashOperator(
        task_id="say_hello_bash",
        bash_command="echo 'Starting the pipeline...'",
    )

    task_python = PythonOperator(
        task_id="say_hello_python",
        python_callable=greet,
    )

    task_end = BashOperator(
        task_id="say_done",
        bash_command="echo 'Pipeline complete.'",
    )

    # Define the order of execution
    task_start >> task_python >> task_end

```

Step 3 — Upload the DAG

Find the environment's DAG bucket:

```
gcloud composer environments describe my-first-composer-e
  --location us-central1 \
  --format="get(config.dagGcsPrefix) "
```

Then upload:

```
gcloud storage cp hello_composer.py <dag-bucket-path>/dag
```

Composer auto-detects new files in `/dags` within a couple of minutes — no restart needed.

Step 4 — Watch it run

Open the Airflow UI (Console → Composer → your environment → "Airflow webserver" link). You'll see:

- **DAGs list view** — toggle the DAG "on" (unpause it) if it's paused by default.
- **Graph view** — visualizes tasks as nodes with the dependency arrows you defined.
- **Grid view** — a timeline of every run, color-coded by status.
- Click any task box → **Logs** to see exactly what it printed (useful for debugging).

Step 5 — A more realistic GCP DAG (BigQuery + GCS example)

```
from airflow import DAG
from airflow.providers.google.cloud.transfers.gcs_to_bigq
from airflow.providers.google.cloud.operators.bigquery im
from airflow.providers.google.cloud.sensors.gcs import GC
from datetime import datetime

PROJECT = "himanshugcpproject"
```

```
BUCKET = "my-training-bucket"
```

```
DATASET = "training_dataset"
```

```
with DAG(
```

```
    dag_id="gcs_to_bq_pipeline",
```

```
    schedule_interval="@daily",
```

```
    start_date=datetime(2026, 7, 1),
```

```
    catchup=False,
```

```
) as dag:
```

```
    wait_for_file = GCSObjectExistenceSensor(
```

```
        task_id="wait_for_input_file",
```

```
        bucket=BUCKET,
```

```
        object="incoming/daily_sales.csv",
```

```
        timeout=600,          # give up waiting after 10 min
```

```
        poke_interval=60,    # check every 60 seconds
```

```
    )
```

```
    load_to_bq = GCSToBigQueryOperator(
```

```
        task_id="load_csv_to_bigquery",
```

```
        bucket=BUCKET,
```

```
        source_objects=["incoming/daily_sales.csv"],
```

```
        destination_project_dataset_table=f"{PROJECT}.{DATASET}.raw_sales",
```

```
        source_format="CSV",
```

```
        skip_leading_rows=1,
```

```
        write_disposition="WRITE_TRUNCATE",
```

```
        autodetect=True,
```

```
    )
```

```
    transform = BigQueryInsertJobOperator(
```

```
        task_id="aggregate_daily_sales",
```

```
        configuration={
```

```
            "query": {
```

```
                "query": f"""
```

```
                    SELECT region, SUM(amount) AS total_sales
```

```
                    FROM `{PROJECT}.{DATASET}.raw_sales`
```

```
                    GROUP BY region
```

```
                """
```

```
            "destinationTable": {
```

```
                "projectId": PROJECT,
```

```

        "datasetId": DATASET,
        "tableId": "sales_by_region",
    },
    "writeDisposition": "WRITE_TRUNCATE",
    "useLegacySql": False,
}
},
)

wait_for_file >> load_to_bq >> transform

```

This single example uses a **Sensor** (waits for a file), a **Transfer Operator** (GCS → BigQuery), and a **BigQuery job operator** — the three building blocks you'll reuse constantly in real pipelines.

6. Monitoring, Debugging, and Operations

- **Task logs:** Click a task instance in the UI → "Log." Composer streams these from Cloud Logging.
- **Cloud Monitoring integration:** Composer environments emit metrics (scheduler heartbeat, DAG parse time, task queue length) viewable in Cloud Monitoring — set alerting policies for "DAG failed" or "scheduler down."
- **Airflow CLI via gcloud:**

```

gcloud composer environments run my-first-composer-en
--location us-central1 \
dags list

```

- **Common beginner pitfalls:**
 - Forgetting `catchup=False` → dozens of historical runs fire at once.
 - Using `PythonOperator` to do heavy data processing directly (Airflow workers aren't meant for big compute — trigger Dataflow/Dataproc/BigQuery jobs instead and let Airflow just orchestrate).

- Hardcoding secrets in DAG code instead of using **Connections**, **Variables**, or **Secret Manager backend** integration.
 - Not making tasks **idempotent** — a re-run should not double-insert data or corrupt state.
 - DAG file top-level code runs every time the scheduler parses the file (every ~30 sec by default) — keep import/setup logic light, and avoid expensive API calls at DAG-definition time.
-

7. Pricing Considerations (High-Level)

Composer billing = GKE compute (Autopilot-style, per environment size/workload) + Cloud SQL (metadata DB) + networking + Cloud Storage (for DAGs/logs). Composer 2/3's autoscaling means idle environments cost less than the old fixed-size Composer 1 model, but a running environment is **never fully free** — even a "Small" environment on standing infrastructure incurs continuous cost, so for pure learning, remember to **delete the environment** when you're done experimenting:

```
gcloud composer environments delete my-first-composer-env
```

8. Suggested Learning Path

1. Create a small Composer 2 environment.
2. Deploy `hello_composer.py`, get comfortable with Graph/Grid views and logs.
3. Deploy the GCS → BigQuery example; modify it to point at your own bucket/dataset.
4. Learn `trigger_rule`, XComs, and branching (`BranchPythonOperator`) to build conditional pipelines.
5. Explore Task Groups (visually grouping related tasks) and Dynamic Task Mapping (Airflow 2.3+, for running a task N times

- over a dynamic list — e.g., one task per file in a folder).
6. Integrate with Dataflow Flex Templates / Dataproc for heavier ETL, using Composer purely as the orchestrator.
 7. Delete the environment when done, or keep it running with a couple of low-frequency training DAGs.
-

Quick Reference Cheat-Sheet

I want to...	Use
Run a shell command	<code>BashOperator</code>
Run Python code	<code>PythonOperator</code>
Wait for a file in GCS	<code>GCSObjectExistenceSensor</code>
Load data into BigQuery	<code>GCSToBigQueryOperator</code>
Run a BigQuery SQL query	<code>BigQueryInsertJobOperator</code>
Trigger a Dataflow job	<code>DataflowStartFlexTemplateOperator</code>
Trigger a Dataproc job	<code>DataprocSubmitJobOperator</code>
Pass small data between tasks	<code>XCom (ti.xcom_push / ti.xcom_pull)</code>
Store a reusable config value	Airflow Variable
Store credentials for an external system	Airflow Connection
Run cleanup regardless of upstream success/failure	<code>trigger_rule="all_done"</code>

This handbook covers Airflow 2.x concepts as implemented in Cloud Composer 2/3. Airflow syntax below version 2.0 (Airflow 1.x, e.g.

provide_context, old-style operator imports) is deprecated — always write new DAGs using the Airflow 2.x/TaskFlow-compatible style shown above.